

Dilemme de l'agent double

- **Limite de temps** : 12 minutes.
- **Stockage** : 5 GB
- **Environnement** : un GPU (≈ 16 GB VRAM), sans internet
- **Taille de la solution** : `solution.ipynb` ≤ 1 MB
- **Score baseline** : 0

Au centre national d'IA d'Astana, deux modèles informatiques — le modèle R (un ResNet-18) et le modèle V (un ViT-Tiny) — analysent des photos. Actuellement, les deux modèles accomplissent un travail parfait, avec une exactitude de 100%, et donnent la même réponse pour chaque image. Pour tester à quel point leurs « cerveaux » intelligents sont réellement différents, le scientifique en chef vous lance un défi : apporter à chaque photo de minuscules modifications de pixels, presque invisibles, afin que le modèle R et le modèle V soient en désaccord complet.



1. Tâche

Deux classifieurs d'images préentraînés examinent la même image. Sur les images fournies dans cette tâche, les deux classifieurs atteignent une exactitude de 100%.

- **Modèle R** : `torchvision.models.resnet18` (un CNN, ResNet18).
- **Modèle V** : `timm.de_vit_tiny_patch16_224` (un Transformer, ViT-Tiny).

Votre tâche consiste à créer une petite modification (« perturbation ») pour chaque image afin que les deux modèles soient en désaccord. Pour chaque image, vous devez créer **deux perturbations différentes** :

- **Type A** : après son ajout, le modèle R classe toujours correctement l'image, mais le modèle V la classe incorrectement.
- **Type B** : après son ajout, le modèle V classe toujours correctement l'image, mais le modèle R la classe incorrectement.

Chaque perturbation doit être suffisamment *petite* pour être difficile à remarquer. Les perturbations plus petites obtiennent un score plus élevé (voir la section 5). La perturbation est appliquée directement à l'image originale, au niveau des pixels.

2. Données publiques

Un ensemble d'images est fourni avec la tâche, organisé en deux splits — `train` (100 images) et `test_public` (100 images) — chacun contenant des images de résolutions variées. Toutes les images appartiennent aux 1000 classes d'ImageNet-1K, et le modèle R comme le modèle V atteignent une exactitude de 100% sur les deux splits.

Les fichiers suivants sont fournis :

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Lors de l'évaluation, votre dossier `dataset/test_public/` est remplacé de manière transparente par deux ensembles cachés d'images (`test_leaderboard_a` et `test_leaderboard_b`) pour le calcul officiel du score. Chacun contient **100 images** au format PNG et un fichier d'étiquettes.

Remarque : pour cette tâche, les étiquettes des datasets de test sont accessibles.

3. Format de sortie

Pour chaque image, vous devez produire deux fichiers :

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), correspond au nom de l'image dans les datasets.
- Chaque fichier est un tenseur unique enregistré avec `torch.save`. Sa forme doit être $3 \times H \times W$, où H et W correspondent à la résolution **originale** de cette image (et non à 224×224).

- Le code ne doit produire qu'un seul fichier ZIP, `submission.zip`. Placez tous les fichiers `.pt` au niveau supérieur de l'archive ZIP, sans dossier englobant ni sous-répertoire.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Le notebook vous avertira si le format de sortie présente des problèmes.

4. Contraintes

- **Modèles** : vous devez utiliser `torchvision.models.resnet18(pretrained=True)` et `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Aucun autre modèle préentraîné n'est autorisé.
- **Pipeline de transformation (imposé lors de l'évaluation)** : `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` pour plus de détails.
- **Résolution de la perturbation** : elle doit correspondre à la résolution **originale** de l'image brute (et non à 224×224). Le tenseur est ajouté à l'image brute *avant* le pipeline de transformation.
- **Format de sortie** : fichiers `.pt` uniquement — aucun PNG/JPG . Les tenseurs sont ajoutés à l'image brute et les valeurs des pixels sont écrêtées dans `[0, 1]` avant le prétraitement.
- **Nommage des fichiers** : liste à plat, au format strict `{index}_a.pt / {index}_b.pt`. Aucun sous-répertoire dans le fichier zip.
- **Bibliothèques** : `torch`, `torchvision`, `timm`.

5. Calcul du score

Le score final est calculé comme suit. Soit M le nombre d'images dans le split, $Score_A$ le nombre de perturbations de type A réussies et $Score_B$ le nombre de perturbations de type B réussies :

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF est une fonction conçue pour pénaliser les perturbations ayant une norme élevée et pour être très sensible près du plafond de performance. Elle est bornée dans l'intervalle allant de 0.5 à 1. L'implémentation complète est présentée dans la section 8 de `solution.ipynb`.

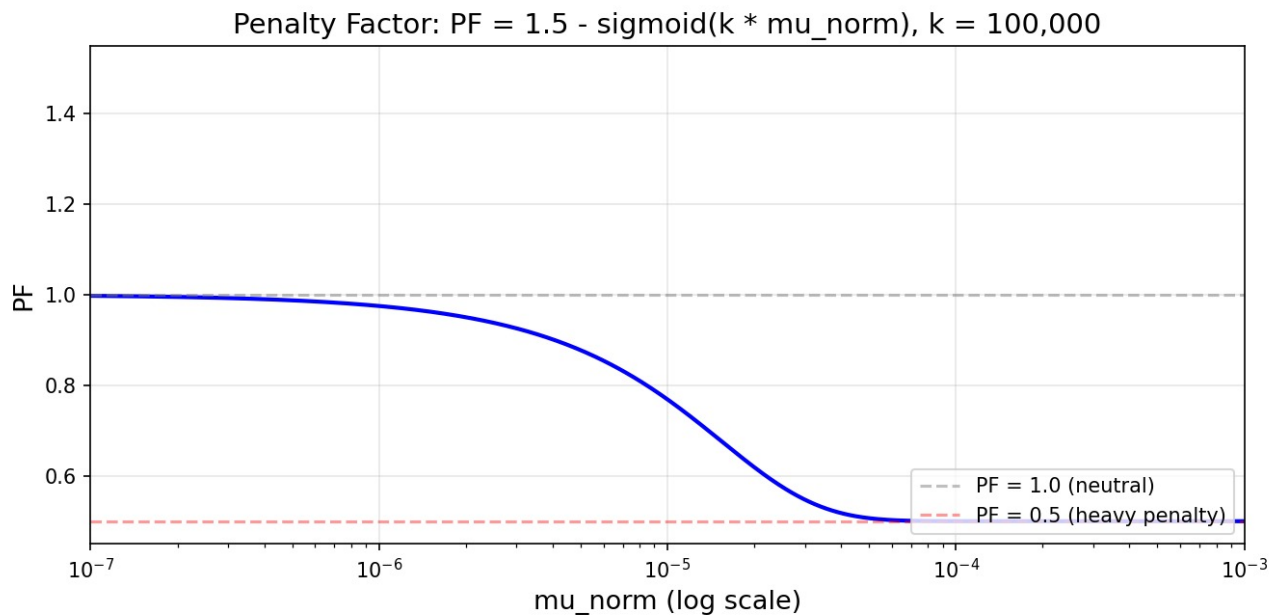


Figure : courbe de la fonction de pénalité.

6. Vérifier la soumission

Le notebook contient des vérifications qui vous avertissent en cas de problèmes de formatage, dans la section 7 du notebook `solution.ipynb`.

7. Tests locaux

`solution.ipynb` contient un exemple complet et fonctionnel. Il charge les données publiques, les deux modèles et le système officiel de calcul du score, puis écrit un fichier ZIP de soumission. Lisez-le avant de commencer.

8. Procédure de soumission

- Enregistrez vos modifications dans `solution.ipynb`.
- Ouvrez l'onglet Git dans la barre latérale gauche de JupyterLab.
- **Stage** `solution.ipynb` (l'icône + située à côté).
- Saisissez un message de commit et cliquez sur **Commit**.
- Cliquez sur l'icône de nuage avec une flèche vers le haut pour effectuer le push.
- Revenez sur cette page du concours et cliquez sur **Submit**.

Soumettez exactement un fichier, nommé `solution.ipynb`.